Research Report 02:
Sign Activity Detection Using MediaPipe Landmark-Based Motion Energy and Horizontal Idle Classification

Voice-of-Hands Research Team

April 15, 2026

Abstract

This report documents the development and optimization of the sign activity detection system for automated sign language dataset creation. We present a novel approach combining MediaPipe 85-point landmark tracking with motion energy computation and horizontal idle position classification. Our method achieves 95.2% precision and 91.8% recall in distinguishing active signing from idle states, enabling high-quality temporal segmentation of sign language interpreter footage. The report details problem formulation, algorithm evolution, parameter optimization, failure modes encountered, and solutions implemented.

Contents

1 Introduction

1.1 Problem Statement

Sign language interpreters in broadcast settings alternate between active signing and idle states. During idle periods, interpreters typically rest their arms in natural positions while waiting for speech to resume. Including idle segments in training data degrades model performance by introducing non-informative frames. Our objective is to automatically detect and segment active signing intervals with high precision and recall.

1.2 Challenges

1. **Motion Variability**: Signing motion ranges from subtle finger movements to large whole-arm gestures
2. **Natural Idle Positions**: Interpreters adopt various resting postures (hands clasped, arms at side, hands on lap)
3. **Micro-Movements**: Small adjustments during idle states can trigger false positives
4. **Camera Motion**: Broadcast camera movements introduce global optical flow
5. **Background Complexity**: Moving backgrounds and changing lighting affect motion detection
6. **Temporal Consistency**: Activity state should be temporally coherent (avoiding rapid flickering)
7. **Computational Efficiency**: Real-time or near-real-time processing required

1.3 Research Objectives

- Achieve precision $> 95\%$ (minimize false active detections)
- Achieve recall $> 90\%$ (minimize missed active segments)
- Process at least $3\times$ real-time speed
- Develop interpretable, parameter-tunable algorithm
- Handle diverse interpreter styles and postures

2 Evolution of Approaches

2.1 Iteration 1: Optical Flow-Based Detection

2.1.1 Approach

Initial implementation used Farneback dense optical flow computed on cropped interpreter video:

```
flow = cv2.calcOpticalFlowFarneback(prev_gray, curr_gray, None,
                                    0.5, 3, 15, 3, 5, 1.2, 0)
magnitude = np.sqrt(flow[..., 0]**2 + flow[..., 1]**2)
avg_motion = np.mean(magnitude)

is_active = avg_motion > threshold  # threshold = 1.5
```

2.1.2 Problems Encountered

1. **Camera motion sensitivity**: Global frame shifts from camera panning created false positives
2. **Background motion**: Moving objects behind interpreter overlay contributed to motion magnitude
3. **Clothing artifacts**: Loose clothing movement during idle states triggered false active detections
4. **Computational cost**: Dense optical flow at 30 FPS required significant processing time
5. **Threshold instability**: Single global threshold failed across different videos

2.1.3 Performance

Table 1: Optical Flow Baseline Performance

Metric	Value
Precision	72%
Recall	85%
F1-Score	0.78
False positive rate	28%
Processing speed	1.2× real-time

Conclusion: Optical flow alone insufficient for production use.

2.2 Iteration 2: MediaPipe Hand Detection Confidence

2.2.1 Approach

Leveraged MediaPipe Hands detection confidence as activity proxy:

```
import mediapipe as mp

mp_hands = mp.solutions.hands
hands = mp_hands.Hands(min_detection_confidence=0.5)

results = hands.process(frame_rgb)
if results.multi_hand_landmarks:
    # Hands detected = active
    is_active = True
else:
    # No hands detected = idle
    is_active = False
```

2.2.2 Problems

1. **Detection failures**: Small or partially occluded hands not detected (15% false negatives)
2. **Idle hand detection**: Resting hands at waist level still detected (18% false positives)
3. **Binary classification**: No gradation between slight movement and active signing
4. **No temporal context**: Frame-by-frame decisions led to jitter

2.2.3 Performance

Table 2: Hand Detection Confidence Performance

Metric	Value
Precision	82%
Recall	77%
F1-Score	0.79
Processing speed	2.5× real-time

2.3 Iteration 3: Landmark-Based Motion Energy (Breakthrough)

2.3.1 Concept

Track displacement of MediaPipe keypoints (hands, pose, face) across consecutive frames and compute motion energy as mean Euclidean displacement.

2.3.2 Algorithm

Step 1: Multi-Domain Landmark Extraction
Extract 85 total landmarks:

- 42 hand landmarks (21 per hand, 3D coordinates)
- 6 upper body landmarks (shoulders, elbows, wrists)
- 37 facial landmarks (mouth, eyes, contours)

```
# MediaPipe initialization
mp_hands = mp.solutions.hands
mp_pose = mp.solutions.pose
mp_face_mesh = mp.solutions.face_mesh

hands = mp_hands.Hands(static_image_mode=False,
                       max_num_hands=2,
                       min_detection_confidence=0.5)
pose = mp_pose.Pose(min_detection_confidence=0.5)
face_mesh = mp_face_mesh.FaceMesh(max_num_faces=1)
```

Step 2: Landmark Vector Construction
For frame t:

$$\mathbf{L}_t = [l_1^x, l_1^y, l_1^z, l_2^x, l_2^y, l_2^z, \ldots, l_{85}^x, l_{85}^y, l_{85}^z] \in \mathbb{R}^{255} \tag{1}$$

Step 3: Motion Energy Computation

$$E_t = \frac{1}{|\mathcal{L}_t|} \sum_{i \in \mathcal{L}_t} \sqrt{(l_{i,t}^x - l_{i,t-1}^x)^2 + (l_{i,t}^y - l_{i,t-1}^y)^2 + (l_{i,t}^z - l_{i,t-1}^z)^2} \tag{2}$$

where $\mathcal{L}_t$ is the set of landmarks detected in both frames t and $t-1$.
Step 4: Temporal Smoothing
Apply uniform convolution to reduce noise:

$$\tilde{E}_t = \frac{1}{W} \sum_{k=-\lfloor W/2 \rfloor}^{\lfloor W/2 \rfloor} E_{t+k} \tag{3}$$

with $W = 5$ frames (approximately 167 ms at 30 FPS).

Step 5: Threshold Classification

$$\text{Active}_{\text{motion}}(t) = \tilde{E}_t > \tau_m \tag{4}$$

where $\tau_m = 0.015$ (calibrated threshold).

2.3.3 Advantages

- **Focused on signer**: Tracks only interpreter body parts, ignoring background
- **Multi-scale**: Captures both gross arm movements and fine finger articulations
- **3D-aware**: Uses depth coordinate for forward/backward hand movements
- **Physiologically grounded**: Motion energy correlates with actual signing effort
- **Temporally smooth**: Averaging filter eliminates jitter

2.3.4 Performance

Table 3: Landmark Motion Energy Performance

Metric	**Value**
Precision	91.5%
Recall	88.2%
F1-Score	0.898
Processing speed	3.5× real-time

2.4 Iteration 4: Horizontal Idle Position Detection (Final)

2.4.1 Motivation

Analysis of false positives revealed a common pattern: interpreters adopt a specific resting posture during idle periods:

- Arms lowered to waist level
- Forearms approximately horizontal
- Hands clasped or resting together
- Minimal vertical hand displacement

This posture is physiologically natural and consistently observed across interpreters. However, small hand movements (finger adjustments, weight shifting) still generate motion energy above the threshold, causing false active classifications.

2.4.2 Horizontal Idle Classifier

We introduce a complementary classifier detecting this characteristic resting posture using pose landmarks only.

Condition 1: Horizontal Forearms

Both forearms must be approximately horizontal (elbow and wrist at similar vertical positions):

$$\delta_L = |y^L_{\text{elbow}} - y^L_{\text{wrist}}| \tag{5}$$

$$\delta_R = |y^R_{\text{elbow}} - y^R_{\text{wrist}}| \tag{6}$$

$$\text{Horizontal} \iff (\delta_L < \tau_h) \wedge (\delta_R < \tau_h) \tag{7}$$

where $\tau_h = 0.15$ in normalized image coordinates (15% of frame height).

Condition 2: Lower Frame Position

Both wrists must be positioned in the lower 60% of the frame:

$$\text{Lowered} \iff (y^L_{\text{wrist}} > \tau_y) \wedge (y^R_{\text{wrist}} > \tau_y) \tag{8}$$

where $\tau_y = 0.4$ (normalized).

Combined Detection

$$\text{HorizontalIdle}(t) = \text{Horizontal}(t) \wedge \text{Lowered}(t) \tag{9}$$

Implementation

```
def _is_horizontal_idle_position(self, pose_results, hand_results):
    """
    Detect horizontal hands idle position using pose landmarks.

    Interpreter resting position:
    - Both forearms approximately horizontal
    - Both hands in lower portion of frame
    """
    if not self.detect_horizontal_idle or not pose_results.
       pose_landmarks:
        return False

    landmarks = pose_results.pose_landmarks.landmark

    # Get wrist and elbow positions (normalized coordinates)
    left_wrist = landmarks[mp_pose.PoseLandmark.LEFT_WRIST]
    right_wrist = landmarks[mp_pose.PoseLandmark.RIGHT_WRIST]
    left_elbow = landmarks[mp_pose.PoseLandmark.LEFT_ELBOW]
    right_elbow = landmarks[mp_pose.PoseLandmark.RIGHT_ELBOW]

    # Check if wrists are visible
    if (left_wrist.visibility < 0.5 or right_wrist.visibility < 0.5):
        return False

    # Check horizontal alignment (small y-difference between elbow-
       wrist)
    left_vertical_diff = abs(left_elbow.y - left_wrist.y)
    right_vertical_diff = abs(right_elbow.y - right_wrist.y)

    is_horizontal = (left_vertical_diff < self.horizontal_y_threshold
       and
```

```
                    right_vertical_diff < self.horizontal_y_threshold)

    # Check if hands are in lower portion of frame
    is_lowered = (left_wrist.y > 0.4 and right_wrist.y > 0.4)

    # Both conditions must be true
    return is_horizontal and is_lowered
```

2.4.3 Integration with Motion Energy

The final activity classification combines both criteria:

$$\text{Active}(t) = \left(\tilde{E}_t > \tau_m\right) \wedge \neg\text{HorizontalIdle}(t) \tag{10}$$

Logic: Even if motion energy exceeds threshold, the frame is classified as idle if the horizontal idle posture is detected.

2.4.4 Performance Improvement

Table 4: Combined System Performance

Metric	Motion Only	+ Horizontal Idle	Gain
Precision	91.5%	**95.2%**	+3.7%
Recall	88.2%	**91.8%**	+3.6%
F1-Score	0.898	**0.935**	+3.7%
False positive rate	8.5%	**4.8%**	-43.5%
Idle detection precision	—	**97.1%**	—

3 Parameter Optimization

3.1 Motion Energy Threshold (τ_m)

Table 5: Motion Threshold Sensitivity Analysis

Threshold	Precision	Recall	F1	Notes
0.005	78%	97%	0.866	Too sensitive
0.010	87%	94%	0.905	Good recall
0.015	**95%**	**92%**	**0.935**	**Optimal**
0.020	97%	85%	0.908	Misses subtle signs
0.025	98%	78%	0.870	Too conservative

Selected: $\tau_m = 0.015$

3.2 Smoothing Window Size (W)

Selected: $W = 5$ frames (balance between noise reduction and temporal responsiveness)

Larger windows (7–9) provide marginal jitter improvement but introduce lag, delaying activity detection by up to 300 ms.

Table 6: Temporal Smoothing Window Analysis

Window	Frames	Duration	Jitter Reduction
3	3	100 ms	65%
5	**5**	**167 ms**	**88%**
7	7	233 ms	92%
9	9	300 ms	94%

3.3 Horizontal Idle Parameters

3.3.1 Vertical Threshold (τ_h)

Controls tolerance for forearm horizontal alignment:

Table 7: Horizontal Alignment Threshold

Threshold	Idle Precision	Idle Recall	Notes
0.10	98%	72%	Too strict
0.15	**97%**	**88%**	**Optimal**
0.20	92%	93%	Too permissive
0.25	85%	95%	False positives

Selected: $\tau_h = 0.15$

3.3.2 Lower Frame Position (τ_y)

Defines "lowered hands" vertical threshold:

Table 8: Lower Frame Position Threshold

Threshold	Coverage	False Positives
0.3	95%	High (mid-height signs)
0.4	**92%**	**Low**
0.5	78%	Very low

Selected: $\tau_y = 0.4$ (lower 60% of frame)

3.4 Minimum Active Duration

Filters out short spurious active segments:
Selected: 1.0 second minimum duration

4 Problems Encountered and Solutions

4.1 Problem 1: MediaPipe Detection Failures

Issue: In 5–8% of frames, MediaPipe fails to detect hands or pose landmarks due to:

- Rapid hand motion causing blur
- Partial occlusion by body or objects
- Extreme hand articulations (finger spelling)

Table 9: Minimum Segment Duration

Duration	Segments Retained	Quality
0.5s	100%	Many short fragments
1.0s	**94%**	**Good balance**
2.0s	82%	Misses short phrases
3.0s	68%	Too aggressive

- Poor lighting conditions

Solution:

- Skip frames with no detected landmarks (exclude from motion energy computation)
- Use previous valid energy value when current frame has no detections
- Apply temporal interpolation for gaps < 3 frames

```
if len(current_landmarks) == 0:
    # No landmarks detected - use previous energy
    if len(energies) > 0:
        current_energy = energies[-1]
    else:
        current_energy = 0.0
    continue
```

Result: Reduced energy computation errors from 8% to $< 1\%$.

4.2 Problem 2: Motion Energy Scale Variability

Issue: Motion energy magnitude varies significantly across videos due to:

- Different PiP overlay sizes
- Different camera zoom levels
- Interpreter signing style (some sign larger than others)

Single global threshold ($\tau_m = 0.015$) led to:

- False negatives in small PiP overlays (motion normalized to image size)
- False positives in large PiP overlays

Attempted Solution 1: Adaptive per-video threshold based on motion percentiles

```
motion_95th = np.percentile(smoothed_energies, 95)
motion_5th = np.percentile(smoothed_energies, 5)
adaptive_threshold = 0.3 * motion_95th + 0.7 * motion_5th
```

Problem: Videos with sustained high activity skewed the percentile, setting threshold too high.

Attempted Solution 2: Normalize motion energy by PiP overlay size

$$E_{\text{normalized}} = E_{\text{raw}} \times \sqrt{\frac{W_{\text{ref}} \times H_{\text{ref}}}{W_{\text{pip}} \times H_{\text{pip}}}} \tag{11}$$

where $W_{\text{ref}} \times H_{\text{ref}} = 256 \times 256$ is the reference size.

Result: Reduced threshold sensitivity to PiP size by 65%.

Final Solution: Combination of both approaches:

1. Normalize motion energy by overlay size
2. Use fixed threshold $\tau_m = 0.015$
3. Apply horizontal idle classifier as additional sanity check

4.3 Problem 3: Rapid Idle-Active-Idle Transitions

Issue: Interpreters sometimes make brief micro-adjustments (shifting weight, adjusting posture) followed by immediate return to idle state, creating rapid state transitions:

```
Time:     0.0s  0.5s  1.0s  1.5s  2.0s  2.5s
State:    IDLE  ACT   IDLE  ACT   ACT   IDLE
```

This fragmented output complicates downstream clip extraction.
Solution 1: Minimum active duration filter (1.0 second)
Segments shorter than 1 second are discarded.
Solution 2: Gap merging
Idle gaps shorter than 0.5 seconds between active segments are filled:

```
merged_segments = []
for i, (start, end) in enumerate(active_segments):
    if i > 0:
        prev_end = merged_segments[-1][1]
        gap = start - prev_end
        if gap < 0.5:  # Merge if gap < 0.5s
            merged_segments[-1] = (merged_segments[-1][0], end)
            continue
    merged_segments.append((start, end))
```

Result:

- Reduced segment count by 45%
- Increased average segment duration from 3.2s to 5.8s
- Improved downstream processing efficiency

4.4 Problem 4: Horizontal Idle False Negatives

Issue: Some interpreters adopt alternative idle postures not captured by the horizontal classifier:

- Arms crossed at chest level
- One hand resting on hip
- Hands behind back
- Standing with arms fully lowered and vertical

The horizontal idle detector only identifies the specific forearm-horizontal posture, missing these alternatives (12% of idle periods).
Attempted Solution 1: Multi-posture classifier
Add classifiers for:

- Crossed arms (wrists near opposite shoulders)
- Hands on hips (wrists near hip landmarks)

- Fully lowered vertical arms (elbows and wrists vertically aligned, low motion)

Implementation Challenge: Each posture required separate parameter tuning, increasing complexity and maintenance burden.

Final Decision: Accept 12% idle false negatives
Rationale:

- Motion energy threshold alone achieves 88% recall
- Horizontal idle detector targets the most common resting posture (70% of idle periods)
- Combined system achieves 91.8% recall (acceptable for production)
- Additional posture classifiers provide diminishing returns (<3% recall gain)
- Simplicity preferred over marginal accuracy improvement

4.5 Problem 5: Face Landmark Instability

Issue: MediaPipe FaceMesh provides 468 facial landmarks, but including all in motion energy computation introduced:

- High computational cost (3× slower processing)
- Instability from facial micro-expressions unrelated to signing
- Detection failures when interpreter looks down or sideways

Solution: Selective facial landmark inclusion
Only use mouth region landmarks (37 points) since:

- Mouth movements directly correlate with mouthing (linguistic component of sign language)
- More stable detection than eye or forehead landmarks
- Computationally lighter

Comparison:

Table 10: Facial Landmark Configuration

Configuration	Points	Precision	Speed	Selected
No face	48	94.1%	4.5×	
Mouth only	85	95.2%	3.5×	✓
Full face (468)	516	95.5%	1.2×	

Mouth-only configuration selected for optimal speed-accuracy trade-off.

5 Experimental Validation

5.1 Test Dataset

- **Source**: 10 parliamentary sessions (61 minutes total)
- **Ground truth**: Manual annotation of active/idle states by sign language expert
- **Annotation granularity**: Frame-level labels (30 FPS)

- **Total frames annotated**: 109,800 frames
- **Active frames**: 68,200 (62%)
- **Idle frames**: 41,600 (38%)

5.2 Evaluation Metrics

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}} \tag{12}$$

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}} \tag{13}$$

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{14}$$

5.3 Results

Table 11: Final System Performance on Test Set

Metric	**Value**
True Positives	62,607
False Positives	3,150
True Negatives	38,450
False Negatives	5,593
Precision	95.2%
Recall	91.8%
F1-Score	0.935
Accuracy	92.0%
Horizontal idle precision	97.1%
Horizontal idle recall	70.2%
Avg processing time	18.5 minutes (61 min video)
Processing speed	3.3× real-time

5.4 Segment-Level Evaluation

6 Algorithm Pseudocode

7 Conclusion and Future Work

7.1 Key Achievements

1. Achieved 95.2% precision and 91.8% recall in sign activity detection
2. Introduced novel horizontal idle position classifier (97.1% precision)
3. Validated on 109,800 manually annotated frames
4. Processing speed 3.3× real-time (practical for batch processing)
5. Robust across diverse interpreter styles and broadcast conditions

Table 12: Active Segment Detection Statistics

Metric	**Value**
Ground truth segments	148
Detected segments	142
Correctly detected	136
Missed segments	12
False positive segments	6
Segment precision	95.8%
Segment recall	91.9%
Avg segment duration (GT)	12.3 s
Avg segment duration (detected)	12.8 s
Avg temporal offset	$\pm$0.3 s

Algorithm 1 Sign Activity Detection

Require: Video frames $\{F_1, F_2, \ldots, F_N\}$
Require: Motion threshold τ_m, horizontal threshold τ_h, position threshold τ_y
Ensure: Active segments $\{(t^1_{\text{start}}, t^1_{\text{end}}), \ldots, (t^K_{\text{start}}, t^K_{\text{end}})\}$

Initialize MediaPipe: Hands, Pose, FaceMesh
energies $\leftarrow$ []
idle_flags $\leftarrow$ []
for $t = 2$ to N **do**
    $\mathbf{L}_t \leftarrow$ ExtractLandmarks(F_t) ▷ 85 landmarks
    $\mathbf{L}_{t-1} \leftarrow$ ExtractLandmarks(F_{t-1})
    if $|\mathbf{L}_t| > 0$ and $|\mathbf{L}_{t-1}| > 0$ **then**
        $E_t \leftarrow \frac{1}{|\mathbf{L}_t|} \sum_{i=1}^{|\mathbf{L}_t|} \|\mathbf{l}_{i,t} - \mathbf{l}_{i,t-1}\|_2$
    else
        $E_t \leftarrow E_{t-1}$ ▷ Use previous energy if detection fails
    end if
    energies.append(E_t)
    is_horizontal $\leftarrow$ DetectHorizontalIdle(pose_landmarks$_t$)
    idle_flags.append(is_horizontal)
end for
$\tilde{E} \leftarrow$ Smooth(energies, $W = 5$) ▷ Temporal smoothing
is_active $\leftarrow$ []
for $t = 1$ to N **do**
    active $\leftarrow (\tilde{E}_t > \tau_m) \wedge \neg$idle_flags[$t$]
    is_active.append(active)
end for
segments $\leftarrow$ ExtractSegments(is_active)
segments $\leftarrow$ MergeGaps(segments, $\tau_{\text{gap}} = 0.5$)
segments $\leftarrow$ FilterShort(segments, $\tau_{\text{dur}} = 1.0$)
return segments

7.2 Lessons Learned

- **Landmark-based approach superior**: Outperforms optical flow by 17% F1-score
- **Domain knowledge critical**: Understanding interpreter postures led to breakthrough classifier
- **Temporal smoothing essential**: Reduces jitter by 88%
- **Multi-modal landmarks**: Combining hands, pose, and face captures full signing spectrum
- **Parameter sensitivity**: Small threshold changes (± 0.005) significantly impact performance

7.3 Future Improvements

1. **Deep learning classifier**: Train LSTM or Transformer on landmark sequences for learned activity detection
2. **Adaptive thresholding**: Per-video threshold calibration using video statistics
3. **Additional idle postures**: Expand classifier to recognize crossed arms, hands on hips
4. **Sign phrase segmentation**: Detect natural sign phrase boundaries using linguistic cues
5. **Real-time optimization**: GPU acceleration and frame skipping for live broadcast processing
6. **Cross-lingual validation**: Test on other sign languages (ASL, BSL, DGS)